KAMOGELO SEKHAOLELO

ASSIGNMENT 2 PRESENTATION

Git Fundamentals &

Azure DevOps

Version Control · Repositories · Pushing Code · Real-World Examples

Intern Assessment — Research Presentation

What We Will Cover

01 What is Git & version control?

02 Key Git concepts explained simply

03 Git workflow commands (all 9)

04 What is Azure DevOps?

05 GitHub vs Azure Repos vs Local Git

06 Pushing code to Azure — full demo

Section 1

What is Git & Version Control?

What is Git?

Git is a free, open-source distributed version control system.

It was created by Linus Torvalds in 2005 — the same person who created the Linux operating system. Git tracks every change made to every file in your project over time. It records what changed, who changed it, and when. This means you can go back to any previous version of your project at any point.

Free & Open Source

Git costs nothing to use and the source code is publicly available. It runs on Windows, Mac, and Linux.

Built for Teams

Designed from the start for multiple developers to work on the same project simultaneously without conflict.

Full History Forever

Every version of every file is permanently stored. Nothing is ever truly deleted — you can always recover previous work.

The Problem Git Solves

Without Git — The Old Way

- Dev A edits login.js and saves
- Dev B also edits login.js and saves
- ✗ **Dev A's 3 hours of work is GONE**
- No way to know what changed
- No safe way to test new ideas
- One mistake can break everything

With Git — The Right Way

- ✓ Every developer has a full history copy
- ✓ Changes are tracked with names & timestamps
- ✓ Work on separate branches — no collisions
- ✓ Roll back to any previous version instantly
- ✓ Experiment safely — branches are free
- ✓ See exactly who changed what and why

Centralized vs Distributed Version Control

Centralized (e.g. SVN)

One central server stores all history
Server goes down → **NOBODY can work**
No internet → no commits or history
Slow, expensive branching

Distributed — Git ✓

Every developer has the **full history locally**
Server offline? Keep working normally
Instant, free branch creation
Examples: Git, Mercurial

Real-World Example

Scenario: A company's SVN server goes down on Friday afternoon. 200 developers cannot work until Monday when IT fixes it.
With Git: The GitHub server has 2 hours of maintenance. All 200 developers keep working, committing locally, and push everything when it comes back. Zero productivity lost.

Section 2

Key Git Concepts — Explained Simply

Repository & Commit

Repository (Repo)

The project folder Git is tracking. Contains all your files plus a hidden .git folder with the entire history.

Two types:
- Local repo — on your machine
- Remote repo — on GitHub / Azure Repos

Commit

A permanent snapshot of your staged changes — like a labeled Save point.

Every commit stores:
- A unique ID (SHA hash)
- Author name & timestamp
- Message describing what changed

Real-World Example

Analogy: A commit is like writing an essay and pressing Save — but each save has a label like "Added introduction" so you can jump back to any version later.
Real world: A developer commits: "Fix patient ID validation — was accepting negative numbers". Six months later when a bug reappears, the team searches history and finds this exact fix instantly.

Branch & Merge

Branch

An independent line of development. You work on a branch without touching the stable main code.

- main / master = stable production code
- feature branch = your new work
- Branches are FREE and instant to create

Merge

Combines changes from one branch into another once a feature is complete and reviewed.

- Git auto-merges changes to different lines
- Merge conflict = same line changed by 2 people
- Conflicts must be resolved manually

Real-World Example

Branch example: Takealot has 30 developers. At any time, 10 feature branches are active: feature/payment-gateway, feature/mobile-checkout, bugfix/cart-total. None affect the live site until reviewed and merged.
Merge example: Dev A adds dark mode (feature/dark-mode). Dev B fixes checkout (bugfix/checkout). Both are merged into main — Git combines them cleanly into one codebase.

Clone, Push & Pull

Clone

Downloads a complete copy of a remote repo to your machine — including all files, all branches, and the full history.

You only clone once, when you first join a project.

Push

Uploads your local commits to the remote repository so your teammates can see your work.

Push after committing work you want to share with the team.

Pull

Downloads the latest commits from the remote into your local repo — keeps your code in sync.

Run every morning before starting work so you have everyone's latest changes.

Real-World Example

New developer, day 1: Their team lead sends the Azure Repos URL. They run git clone and within seconds, their laptop has the full codebase including 2 years of commit history — ready to start coding.
Daily routine: Every developer starts with git pull to get overnight changes from colleagues, then pushes at end of day with git push to share the day's work.

Section 3

Git Workflow Commands — All 9 Explained

For each command you will see: what it does · why it exists · when to use it

Commands 1 & 2 — Starting & Staging

`git init`

What: Creates a brand-new empty Git repository in your current folder. Generates a hidden .git folder.
Why:
Without this, Git has no way to track your project.
When:
Run ONCE when starting a brand-new project that isn't already a Git repo.

`git add .  (or git add filename)`

What: Stages all modified/new files — marks them as ready to be included in the next commit.
Why:
Git doesn't auto-track every change. You choose what goes into each commit.
When:
After making changes you want to save. Tip: git add login.js stages ONLY that file.

Real-World Example

git init: A student creates a folder called my-website and runs git init. Git now tracks this project — the .git folder appears (hidden).
git add: A developer edits login.js, style.css, and readme.md but only wants to commit login changes. They run git add login.js — only that file is staged. This keeps commits focused and meaningful.

Commands 3 & 4 — Saving & Branching

`git commit -m "describe what you did"`

What: Saves a snapshot of all staged changes permanently with a descriptive message.
Why:
Commits are your permanent save points. The message tells your team WHY the change was made.
When:
Once staged changes are working and tested. Write clear messages — your team depends on them.

`git branch feature-name`

What: Creates a new branch with the given name, branching off from your current position.
Why:
Branches keep your work isolated — you can experiment freely without touching stable main code.
When:
When starting any new feature, fix, or experiment. Name branches clearly: feature/dark-mode, bugfix/login.

Real-World Example

Good commit message: "Fix null pointer exception in user authentication" — tells the team exactly what was fixed and where.
Bad commit message: "fix" or "changes" — tells nobody anything. At banks like Absa or Standard Bank, commit messages are reviewed by auditors after production incidents.

Commands 5 & 6 — Switching & Merging

`git checkout branch-name  (or -b to create+switch)`

What: Switches your working directory to the specified branch. All your edits now go to that branch.
Why:
You need to be on the right branch before making changes — edits go to whichever branch is active.
When:
After creating a branch, or when switching between tasks. git checkout -b feature-name creates AND switches.

`git merge branch-name`

What: Brings changes from the named branch into your currently active branch.
Why:
This is how completed work gets back into the main codebase after review and testing.
When:
When a feature is complete and approved — switch to main first, then run git merge feature/your-feature.

Real-World Example

Scenario: A Vodacom developer builds feature/reward-points for 2 weeks. The main branch stays untouched the whole time.
After review: They run: git checkout main → git merge feature/reward-points. The new feature is now live in main. The feature branch is then deleted.

Commands 7, 8 & 9 — Connecting to Remote

`git remote add origin <url>`

What: Links your local repository to a remote one (Azure Repos, GitHub). 'origin' is the standard alias.
Why:
Without this link, Git doesn't know where to push or pull from.
When:
Run ONCE when first connecting a local repo to a remote — right after git init on a new project.

`git push origin main`

What: Uploads your local commits to the remote repo.
Why:
Shares your work with the team.
When:
After committing work you want the team to see.

`git pull origin main`

What: Downloads & merges the latest remote changes locally.
Why:
Keeps your copy in sync with the team.
When:
Every morning before you start coding.

Real-World Example

Enterprise rule: At Absa Bank, nobody pushes directly to main. They push to a feature branch — an automated pipeline runs 500 tests. Only after tests pass and 2 seniors approve does it merge into main.
Daily habit: Run git pull every morning. Without it, you are coding on an outdated copy and creating merge conflicts for your teammates.

Section 4

What is Azure DevOps?

Azure DevOps — 5 Core Services

Microsoft's platform for managing the complete software development lifecycle — from planning to deployment.

Azure Repos	Azure Pipelines	Azure Boards	Azure Test Plans	Azure Artifacts
Git-based source code storage & version control	Automated build, test & deploy (CI/CD)	Sprint & task management (Kanban / Scrum)	Manual & automated testing management	Package management (npm, NuGet, Maven, PyPI)
Store all app code	**Auto-deploy on merge**	**Track who builds what**	**Run tests before release**	**Share internal libraries**

Real-World Example

Pick n Pay IT: Azure Boards tracks 200+ tasks. Repos stores all code. Pipelines runs 500 unit tests on every push — deploys to production automatically only if all tests pass. A bug fix written at 9am can be live by 9:30am with full traceability.

Azure Repos & Git — How They Work Together

Azure Repos is NOT a replacement for Git. It is simply a remote host for your Git repository.

Same Git Commands

You use the exact same git add, git commit, git push commands whether you are pushing to GitHub or Azure Repos. Azure Repos is just a different remote URL — nothing else changes.

Pull Requests

Azure Repos adds a Pull Request (PR) workflow. Instead of pushing directly to main, you push to a feature branch and open a PR. Team members review, comment, and approve before merging.

Permissions & Security

Azure Repos uses Azure Active Directory (AD) — the same login your company uses for email. Fine-grained permissions control who can read, push, or approve merges.

Real-World Example

In practice: A developer at Nedbank writes code exactly the same way as a developer using GitHub. The only difference is the remote URL they push to: https://dev.azure.com/nedbank/... instead of https://github.com/nedbank/...
Azure adds: Branch policies, required reviewers, automated pipelines that run on every push, and integration with Azure Boards to link commits to specific work items (tickets).

Section 5

GitHub vs Azure Repos vs Local Git

Local Git vs GitHub vs Azure Repos

	Local Git	GitHub	Azure Repos
Where it lives	Your machine only	GitHub cloud	Microsoft Azure cloud
Works offline?	Yes — fully	No — needs internet	No — needs internet
Best for	Personal / solo projects	Open source, community	Enterprise, Microsoft stack
CI/CD built in	No	GitHub Actions	Azure Pipelines
Access control	N/A	Teams & org roles	Azure Active Directory
Pricing	Free (always)	Free + paid tiers	Free up to 5 users
Used by	Every developer	Linux, React, Python	Nedbank, MTN, Discovery

Section 6

Pushing Code to Azure Repos — Full Process

Pushing to Azure — Steps 1 to 3

1

Create Azure DevOps Project

Go to dev.azure.com → Sign in with Microsoft account → Click 'New Project' → Give it a name (e.g. retail-pos-system) → Set visibility to Private → Click Create.

2

Create a Repository

Inside your project → Navigate to Repos (left menu) → Click New Repository → Name it (e.g. pos-backend) → Select Git as type → Click Create. Copy the repo URL that appears.

3

Clone the Repository Locally

On your machine, open a terminal and run the clone command. This creates a local folder already linked to Azure.

```
git clone https://dev.azure.com/yourorg/retail-pos-system/_git/pos-backend
```

Pushing to Azure — Steps 4 to 7

4

Make Changes Locally

Navigate into the cloned folder and edit or create files using VS Code or any code editor.

```
cd pos-backend   # then edit your files
```

5

Stage Your Changes

Tell Git which changes should go into the next commit. The dot (.) stages all changed files.

```
git add .
```

6

Commit Your Changes

Save a permanent snapshot with a clear message describing what you did.

```
git commit -m "Initial setup — add base project structure"
```

7

Push to Azure Repos

Upload your commit to Azure. Your code is now visible to the entire team in Azure Repos.

```
git push origin main
```

Real-World Scenario — Junior Developer, Day 1 to Day 3

Day 1

Clone the repo

New developer joins a logistics company. Team lead shares the Azure Repos URL. Developer runs clone — instantly has the full codebase.

```
git clone https://dev.azure.com/logico/fleet-system/_git/api
```

Day 2

Pull + create branch + write code

Before writing a single line, pull the latest code. Then create a dedicated branch for the GPS tracking feature.

```
git pull origin main
git checkout -b feature/driver-tracking
```

Day 3

Stage, commit & push

Feature is done. Stage, commit with a clear message, and push to Azure. Team lead reviews the code in Azure Repos and approves the pull request. Azure Pipelines runs automated tests, then merges to main.

```
git add .
git commit -m "Add real-time GPS tracking endpoint for drivers"
git push origin feature/driver-tracking
```

Key Takeaways

Git solves a real problem

Before Git, teams overwrote each other's work with no recovery. Distributed control gives everyone a full local copy — work continues even offline.

Commits are your save points

Use git add → git commit with clear messages. Your commit history is a permanent, searchable record that teams and auditors rely on.

Branches keep work safe

Always develop in a branch. Merge into main only after testing and team review. This is standard practice at every professional company.

Azure DevOps is a full platform

Azure Repos is one of five services. It uses standard Git commands — the only difference is the remote URL. Azure adds security, CI/CD, and team management on top.

Thank You